



Product Overview & Design Brief

Documenting websites at the source

An onboarding read for anyone joining the project — product, engineering, and design.

July 2026 · v1 (beta) · open-source, single-maintainer

Purpose. This document orients anyone joining document.io to what the tool is, why it exists, and where it is heading. Sections 1–7 provide that grounding; Section 8 sets out the product requirements; Section 9 defines the design priorities, weighted toward the companion browser extension; and Section 10 lists the open questions. It is based on the product as it works today and the document.io and document.io-companion repositories.

1 In one line

document.io lets you document a website *on the website* — attaching written notes to the actual elements of a live page — instead of capturing screenshots or recording videos that drift out of date.

2 The problem

When someone documents a web UI or writes a how-to — onboarding a teammate, producing a support tutorial, capturing a workflow — they reach for screenshots, screen recordings, or a doc that describes the interface in words. All three share one flaw: they are a copy of the interface, detached from it.

The moment the product changes — a button moves, a label is renamed, a screen is redesigned — the screenshots are wrong, the video is stale, and nobody notices until a reader is already confused. Keeping documentation current means re-capturing everything by hand.

For support and onboarding teams — who produce the most how-to content and whose content ages the fastest — this is a constant, invisible tax.

3 The insight

Anchor the documentation to the live page itself. A note isn't a screenshot of the “Export” button — it is attached to the Export button, identified by its place in the page. Open that page with the companion and the notes appear in context, on the real, current UI.

There is nothing to re-screenshot when the surrounding design changes; the annotation still points at its element. This “document at the source” idea is the whole product — everything else serves it.

4 Vision

Documentation that lives on the product, not beside it.

Any web product carries a layer of living documentation that its team can author in place and its users can summon in context. Writing a how-to becomes as fast as walking through the flow once and leaving notes behind. And because the notes are bound to the interface rather than to a snapshot of it, they stay useful as the product evolves — shrinking the gap between “the product changed” and “the docs caught up” toward zero.

5 Who it's for

Primary — the support / onboarding author

Roles: support lead, customer-onboarding specialist, technical writer, developer advocate, solutions engineer. They create tutorials, how-to guides and onboarding walkthroughs for a web product — their own company's app, or a third-party tool their users depend on.

Job to be done: *“When I need to teach someone how to do X in a web app, I want to capture the steps directly on the real screens — so the guide is accurate, fast to make, and doesn't rot when the UI changes.”*

What they feel today: screenshots are tedious to capture and annotate, recordings can't be skimmed or edited, and everything they make silently expires.

Secondary personas

- **Product / UX teams** leaving design feedback and specs on a staging build.
- **QA / dev teams** filing change requests and bug notes anchored to the exact element.
- **Individuals** capturing a personal how-to for later (“how I did X on GitHub”).

The reader / consumer

The person who uses the documentation later — often a customer or new hire, not the author. Today, consuming a doc means opening the target page with the extension in view mode. That reading experience is underserved and is treated here as a first-class design concern (see §9).

6 Strategy

This is a personal, open-source portfolio project, and the strategy is scoped to match — no monetization, no sales motion, no enterprise checklist. Success looks like:

- **A tool that's genuinely pleasant to use** for the primary persona's core loop — not just a tech demo.
- **A crisp, self-evident story** — someone lands on the repo or a two-minute demo and immediately gets it.
- **Modest organic traction** — GitHub stars, a handful of real users, maybe outside contributors: evidence the idea resonates.
- **A portfolio signal** — end-to-end product thinking and execution: a real problem, a clear insight, a polished build.

Positioning

The nearest reference points are Scribe (auto-captured step guides), Loom (screen recordings), and plain screenshots pasted into Notion or Confluence. document.io's distinct claim among them is that it is in-place and durable — the docs sit on the live UI and survive UI change. Lean into that; don't try to out-feature the incumbents.

Guiding principles (KISS / YAGNI)

- **Single-user first.** Real-time collaboration, teams and permissions are explicitly later — don't let them shape the product now.
- **Capture must be effortless.** The core loop (open page → mark an element → write → save) is the product. A second or a click removed here matters more than any feature added elsewhere.
- **Fight staleness by design.** Anything that makes annotations more resilient to UI change — or that surfaces when they've broken — is on-strategy.
- **Meet the page where it is.** The companion is a guest on someone else's arbitrary, sometimes hostile page. Coexisting cleanly is a hard constraint, not a nicety.

Non-goals (for now)

Monetization / pricing, accounts / SSO / orgs, multi-user real-time editing, mobile browsers, analytics dashboards, AI generation. Noted so they don't creep into the redesign.

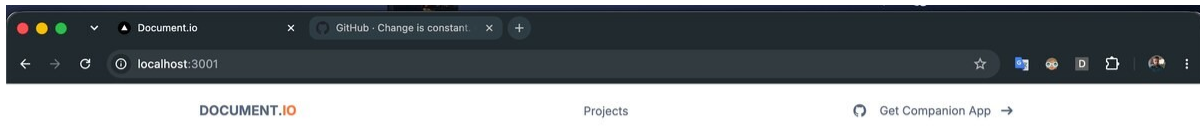
7 How it works today

Two surfaces, one data model.

The Hub — a web app for organizing and opening documentation

A simple three-level hierarchy:

- **Project** — a container for related documentation (e.g. “DVP Onboarding”, “CSS Testing”). Has a name, description and active / inactive status.
- **Documentation** — a single guide within a project, pointed at a target URL (e.g. “Renaming a repository on GitHub”). Opening one launches the target site with the companion active.
- **Annotation** — an individual note, anchored to one element on a page within that documentation.



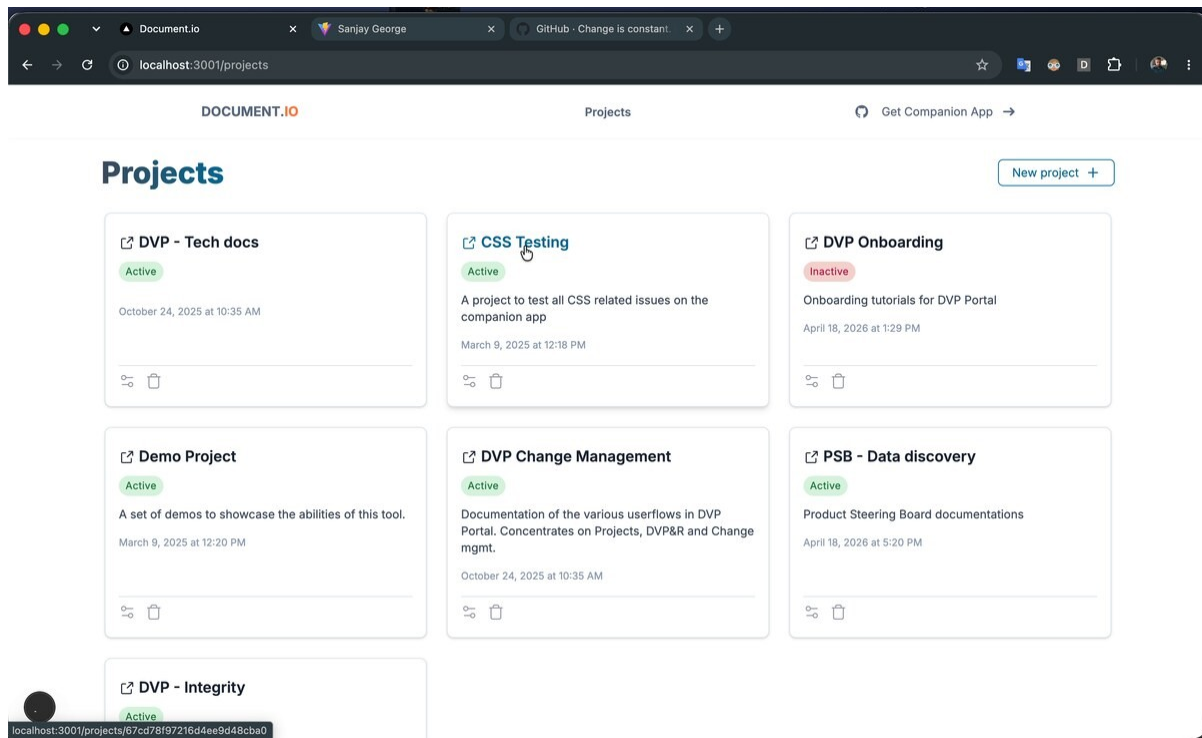
Your Web Documentation Hub

Document.io lets you easily capture and annotate web pages with notes and voice memos. Simplify collaboration and keep your team aligned with clear, accessible documentation.

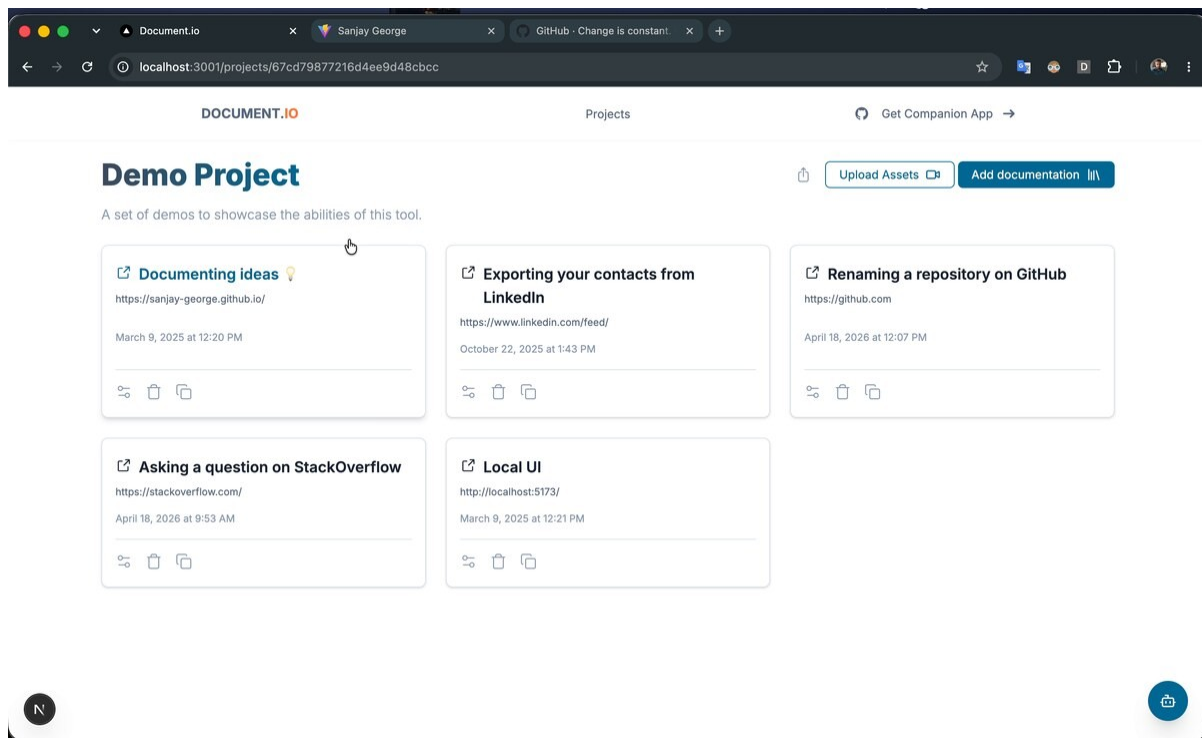
Get Started →



The Hub landing page. “Get Companion App” points users to the browser extension.



Projects list — each card is a container of documentation; “New project” creates one.



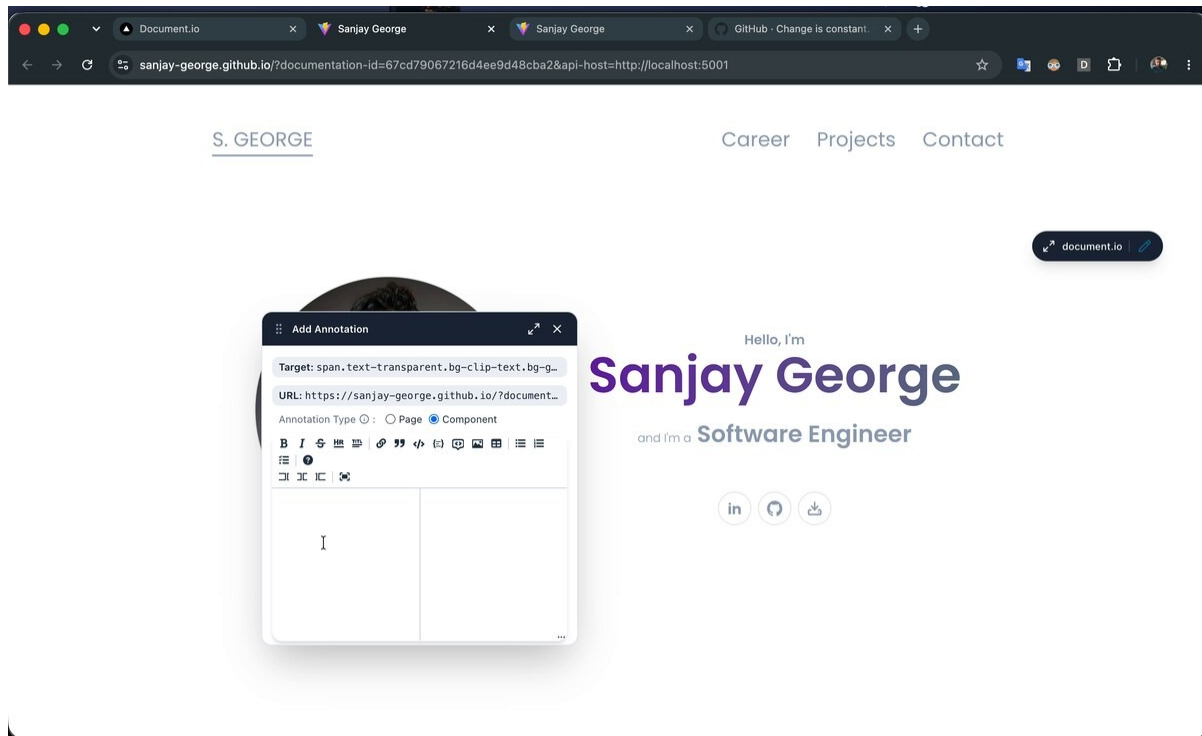
Inside a project — a list of documentation, each targeting a real URL. “Add documentation” starts a new one.

The Companion — a Chrome / Edge extension (the heart of the product)

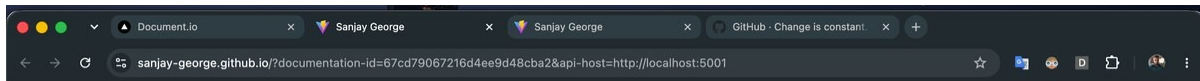
When you open a documentation, the target site loads with the extension injected. A floating pill (“document.io”, bottom-right) is the control: it toggles between view and edit modes and expands the panel.

To author: pick an element on the page and an *Add Annotation* popup opens, pre-filled with the element's target (a CSS selector) and the page URL. You choose a type — Page or Component — and write the note in a rich Markdown editor (headings, bold, lists, quotes, code, tables, images). Save, and the element is now annotated; annotated elements are highlighted on the page.

To read: in view mode a side panel lists the annotations, with tabs for “On this page” vs “All” in the documentation, and renders each note (target, source URL, formatted body). The panel can be minimized while you work.



Authoring — the *Add Annotation* popup. Target selector + URL are captured automatically; Page / Component type; full Markdown toolbar.



S. GEORGE

Career Projects Contact

document.io

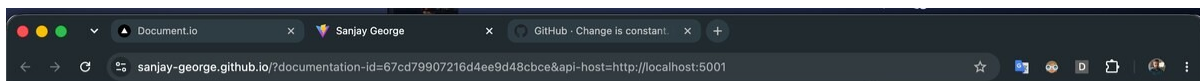


Hello, I'm
Sanjay George

and I'm a **Software Engineer**



An annotated element highlighted in place on the live page.



S. GEORGE

Career Project



Hello, I'm
Sanjay Geo

and I'm a **Software Engin**



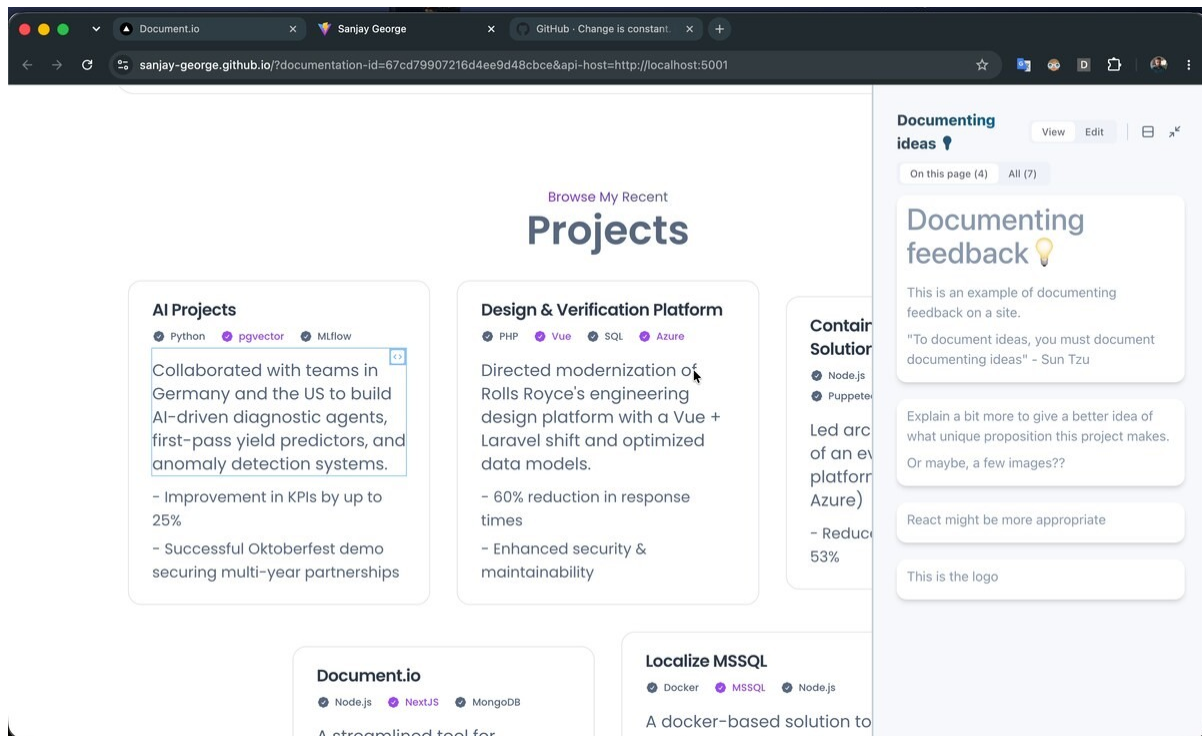
Documenting feedback
Target: #desktop-nav
URL: https://sanjay-george.github.io/

Documenting feedback 💡

This is an example of documenting feedback on a site.

"To document ideas, you must document documenting ideas" - Sun Tzu

Reading — a single annotation in the side panel, showing its Target, source URL and rendered Markdown.



The panel in view mode with "On this page (4)" / "All (7)" tabs and a View / Edit toggle.

Under the hood (context, not something to design)

A background service worker proxies API calls to the Hub's backend, so the tool can reach the API from any page and handle auth and errors centrally. A content script injects the panel UI and watches for SPA navigation (History API changes) so annotations follow client-side route changes. The Hub's built UI is bundled inside the extension, and the target documentation and API host are passed as URL parameters.

The practical implication for design: the pill and panel live inside pages you don't control, and the tool must keep up with pages that change route without ever reloading.

8 Product requirements

Scoped to a single-user, portfolio-stage product, and weighted toward the companion. **P0** = the core loop must work and feel good. **P1** = strongly wanted; shapes the redesign. **P2** = nice, only if cheap.

Companion (extension) — the focus

Requirement	Priority
Enter a documentation and see its existing annotations rendered in context on the live page.	P0
Toggle cleanly between view (read) and edit (author) modes.	P0
Select any element and create an annotation anchored to it, with the selector + URL captured automatically.	P0
Write notes in Markdown with a formatting toolbar; save, edit and delete them.	P0
Visibly indicate which elements are annotated; move between an indicator and its note both ways.	P0
The pill + panel must not break, obscure, or be restyled by the host page; must be movable and minimizable.	P0
Browse annotations in a side panel with “on this page” vs “whole documentation” views; jump from a list item to its element.	P1
Follow SPA route changes so annotations appear / disappear with the client-side page.	P1
Handle the “anchor no longer matches” case gracefully — tell the user an element can't be found rather than failing silently.	P1
Design first-run and empty states: what the panel says with zero annotations, and a clear path to the first one.	P1
Image / video / voice-note annotations (the repo's stated “coming soon”).	P2
Order annotations into a guided, numbered sequence (a “tour”) — high value for tutorials specifically (see §9).	P2

Hub (web app)

Requirement	Priority
Create / list / open projects and documentation; each documentation carries a target URL.	P0
Clear hand-off Hub → companion: open a documentation and land on the target page with the panel ready.	P0
Obvious install / first-run path for the extension, and a sane state when a documentation is opened without it installed.	P1
Basic organization: rename, set active / inactive, delete, and search / filter as the list grows.	P1
Share a documentation via read-only link — acknowledging that reading needs the extension today (a friction point to design around).	P2

Cross-cutting

Requirement	Priority
The whole open → annotate → save → reopen loop is reliable and fast.	P0
A coherent visual identity shared across Hub and companion (today they read as two different apps).	P1
Accessibility of the injected UI (keyboard, focus, contrast) given it sits over unpredictable pages.	P1

9 Design priorities

The areas where design work adds the most value, in order. The effort is weighted toward the companion, where the core experience lives.

1. Redesign the companion's core loop (the #1 job).

The open → select element → write → save → read loop is the product; it should feel effortless and unmistakably in-context. Look closely at:

- The element-selection interaction: how you pick a target, how nested / ambiguous elements are handled, and how you know what you're about to annotate.
- The Add Annotation popup: it carries a lot at once (target, URL, Page / Component, a full Markdown toolbar). What's essential in the moment of capture vs. what can come later? Can capture feel lighter?
- The Page vs Component distinction: it's exposed to the user but its meaning isn't self-evident. Make it obvious — or hide it.
- Modes: view vs edit is a core concept surfaced through a small pill. Is that the right model, and the right control?

2. Design the reading experience as a first-class thing.

Consuming a guide today means opening the page with the extension in view mode and reading the side panel. For support / onboarding, the reader is often a customer or new hire, not the author. How does a reader move through a multi-step guide in order? How do panel notes and on-page highlights guide attention together? What's the least-friction way for a reader to even get into this view?

3. Make the companion a good guest on hostile pages.

The pill and panel render on arbitrary third-party sites with their own CSS, z-index wars and layouts. Design for robustness: placement that rarely collides, a panel that docks without fighting the page, highlights that are visible but not obnoxious, and graceful behavior on very dense or very sparse pages.

4. Unify the visual identity.

Hub and companion currently look like two products. One coherent, simple, modern identity across both — with the constraint that the companion's styling must be fully isolated from whatever page it's injected into.

5. Nail the empty, first-run and broken states.

First time opening a documentation; a page with zero annotations; an annotation whose element can no longer be found (the staleness case). These states carry the product's promise and are currently undefined.

6. Consider “steps / tours” for tutorials (exploratory).

The primary use case — how-to guides — is inherently sequential, yet annotations are today a set, not a sequence. A lightweight way to order them into a numbered walkthrough could make document.io dramatically better at its main job. Flagged as an idea to pressure-test, not a settled requirement (YAGNI — only if it earns its place).

Constraints to design within

- **Chrome / Edge extension (Manifest V3):** the panel is injected via a content script; its styling must be isolated (Shadow DOM or equivalent) and coexist with unknown host CSS.

- **Single-user, no accounts for now:** don't design flows that assume teams, sharing or real-time collaboration.
- **Markdown is the note format;** the editor already exists.
- **Must survive SPA navigation:** the panel's contents update as the route changes, without a reload.
- **Keep it simple:** a smaller, sharper set of well-designed flows beats a broad feature surface.

10 Open questions

Decisions that would sharpen the design and scope:

1. **Reader reality:** is it acceptable long-term that reading a doc requires installing the extension, or is a lightweight read-only web view eventually in scope? (Changes how hard we lean on the extension for consumption.)
2. **Steps / tours:** worth prototyping sequencing now, or keep annotations as an unordered set for the current version?
3. **Page vs Component:** what's the intended distinction — and does the user need to care, or can the system infer it?
4. **Third-party vs own app:** is the headline use case documenting your own product, or any website (“how to do X on GitHub”)? Both work, but the emphasis nudges defaults (staging URLs & auth vs. public sites).
5. **Scope of this redesign:** companion only, or Hub + companion together? (Working assumption: both, companion-weighted.)

Source material: the product as it works today and the [document.io](#) and [document.io-companion](#) repositories.